

DATA266 Lab 1 — Team 5

DATA266 Lab 1 — Team 5

Course: DATA266

Team: 5

Members: Sneha Singh · Ritika Mukesh Neema

Repo: https://github.com/snehas-SJSU/Data_266-Lab1

This is the report source. My sections are filled from the runs that are already on git. Ritika's sections and the comparison tables are left open on purpose. The Kaggle public score is still blank because I have not uploaded `submission.csv` yet. We will turn this file into a PDF after her parts and that score are in.

How we split the work

Part	Sneha Singh	Ritika Mukesh Neema
1 — GPT (TinyStories)	My model, full GPU run	Her model
2 — Yelp polarity	My three models, 100k run	Her three models
3 — CycleGAN	My train, local FID, my audit column	Her train and her audit column
Report / Drive / Kaggle	My half	Her half

We share the raw data. I only write under `sneha_singh/`. She only writes under `ritika_mukesh_neema/`.

Part 1 — GPT from scratch

1.A Sneha Singh

I trained a small character-level GPT on TinyStories. The attention code is mine. I did not use `nn.Transformer` or `nn.MultiheadAttention`.

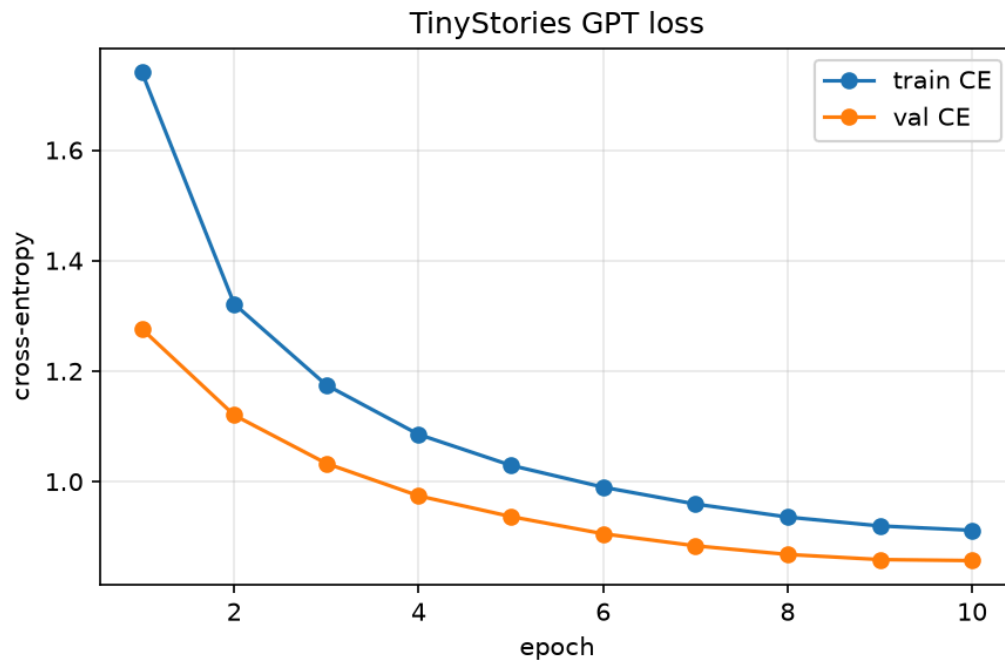
The tokenizer is a simple `char_to_idx / idx_to_char` map, not BPE. The model has 4 layers, 4 heads, embedding size 256, and a context of 128 characters. A causal mask stops each position from seeing the future. I tied the token weights the usual way and trained with AdamW. Bias and LayerNorm do not get weight decay.

The reported run is the full one, not the old 2-epoch smoke test. I took my own split from `TinyStories-train.txt`: 100,000 train windows and 10,000 validation windows, seed 670. That text file is about 1.92 GB, so it is on Drive and not in git: https://drive.google.com/file/d/1K8wGXMKDaLaVdv7ISMNihMH33pGig-CN/view?usp=share_link

I trained for 10 epochs on an NVIDIA GeForce RTX 5090 with mixed precision. Learning rate was 0.001, with 100 warmup steps and then cosine decay. Batch size was 32. The run took 457 seconds and peaked at about 349 MB. There were no NaN losses. The checkpoint `best.pt` is 12 MB, so it is on git.

Metric	Value
Train cross-entropy	0.912
Val cross-entropy	0.857
Perplexity	2.36
Next-character accuracy	0.729
Parameters	3.25M

The loss falls smoothly and the validation curve stays close to the training curve. That is what I wanted from 10 epochs on this size of model.



Part 1 training and validation loss

The prompt I used for samples was “some changes. She added some nice colors”.

Greedy decoding writes a readable start, then gets stuck: “You are very happy. You are very happy.” It also doubles a word, “them them”. Temperature 0.8 does not loop as hard, but the story wanders from colors to a box, a door, and then a tree. I expected some of this. The model only sees 128 characters at a time, and it has no idea where a word ends. The three cases are written up in `task1_llm/sneha_singh/failure_analysis.md`.

1.B Ritika Mukesh Neema

Ritika: architecture, hardware, metrics, a loss plot, a sample, and three failure cases from task1_llm/ritika_mukesh_neema/.

1.C Comparison

Fill this after Ritika's Part 1 is in. Do not compare until both numbers are real.

	Sneha	Ritika
Layers / heads / emb	4 / 4 / 256	
Val CE / perplexity	0.857 / 2.36	
What is different		

Part 2 — Yelp polarity

2.A Sneha Singh

This is Yelp polarity, not IMDB. I learned the embeddings from scratch. I did not use a pretrained language model.

I wanted one simple model and two that can use word order, so a strong baseline would be obvious if the fancier models did not beat it.

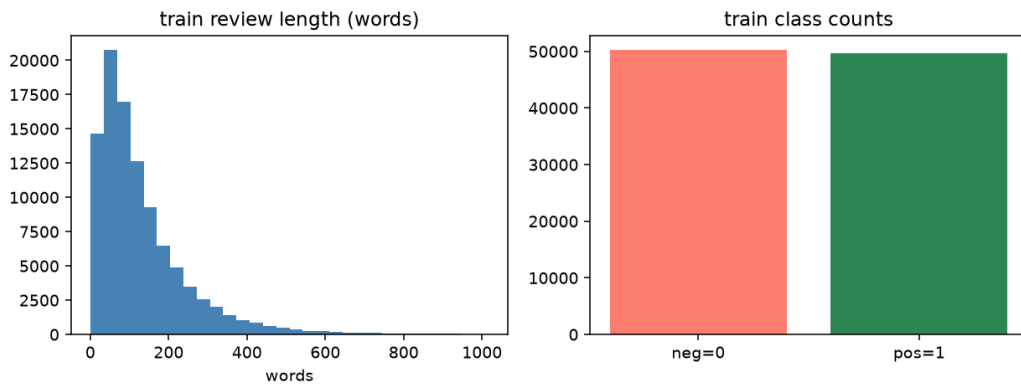
1. **Baseline.** Mean pool over the embeddings, then a linear layer.
2. **BiLSTM.** Reads the review in both directions, so negation and longer sentences have a chance.
3. **TextCNN.** Filters of width 3, 4, and 5, meant to catch short phrases like “not good”.

I lowercased, stripped punctuation, removed stopwords, and lemmatized. The vocabulary comes from the training text only. I sampled 100,000 training reviews. Eleven were empty after cleaning, so the split is 99,989 / 10,000 / 10,000. Five epochs, batch 64, embedding size 100, maximum length 128. All three models ran on my Mac: Apple M4, 16 GB unified memory, MPS. The CUDA peak-memory counter stays 0 on MPS, so I left that field at 0.0 instead of inventing a number.

Model	Acc	Macro-F1	ROC-AUC	MCC	Brier	ECE	Time
Baseline	0.923	0.923	0.973	0.845	0.059	0.011	114 s
BiLSTM	0.918	0.918	0.975	0.837	0.060	0.017	1706 s
TextCNN	0.921	0.921	0.975	0.841	0.061	0.024	108 s

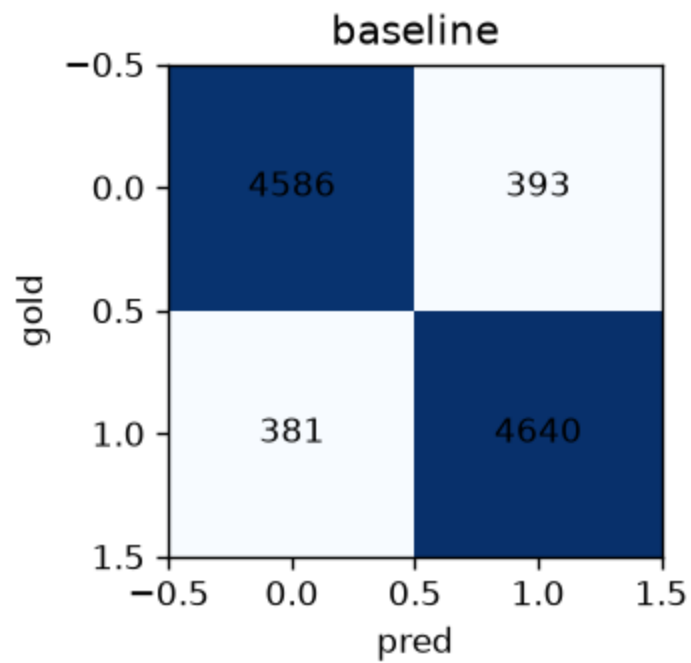
The baseline still has the best macro-F1. McNemar against the baseline is $p = 0.091$ for the BiLSTM and $p = 0.424$ for the TextCNN. On this 10k test set I cannot say the other two are clearly different. Short reviews are a bit easier than long ones for every model. Long-review macro-F1 is 0.917, 0.912, and 0.915 for baseline, BiLSTM, and TextCNN.

The length and class balance of the sample:

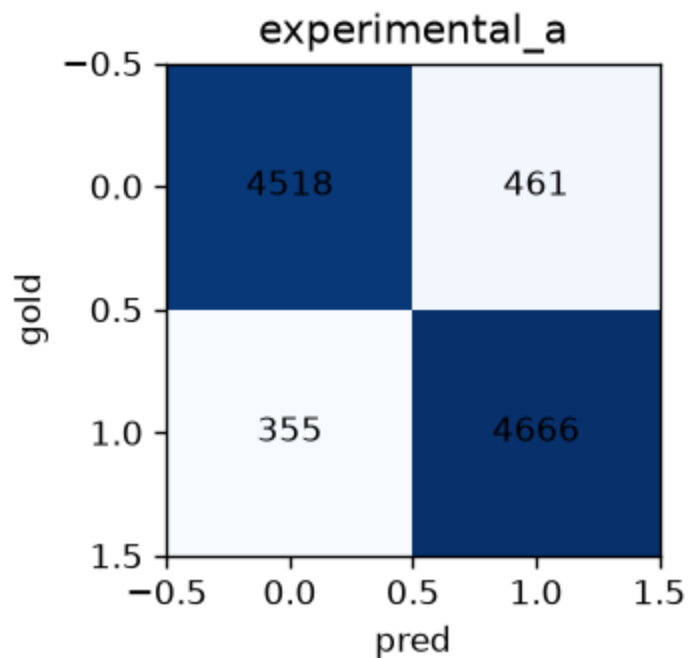


Yelp length and class balance

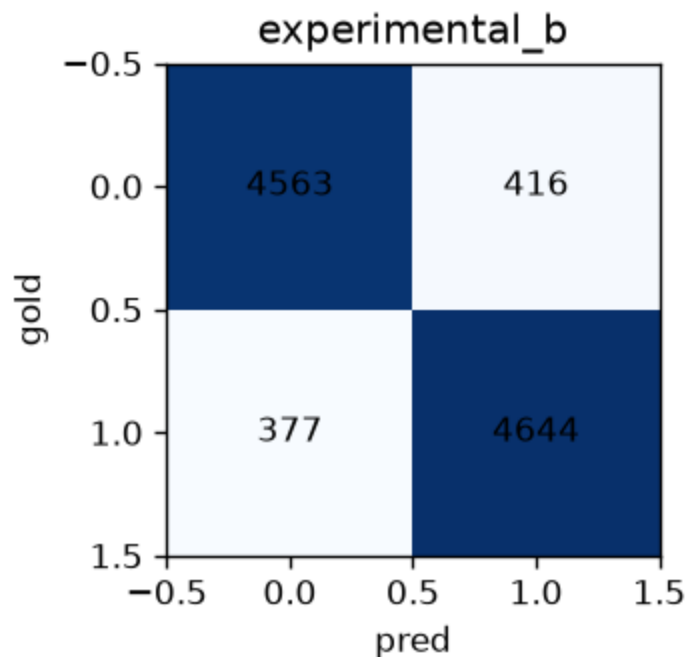
Confusion matrices on the test set:



Baseline confusion matrix



BiLSTM confusion matrix



TextCNN confusion matrix

I read 20 baseline mistakes. The ones that show up most are tiny blurbs the model treats as positive (“food always good”), negation or mixed wording (“not restaurant closed”), and long reviews where the stars and the sentences pull apart. The full list is in `task2_sentiment/sneha_singh/failure_analysis.md`.

This is 100k reviews out of about 560k, not the whole Yelp train file. Cutting reviews at 128 tokens hurts the long ones. I kept negation words, and mixed reviews are still

hard.

2.B Ritika Mukesh Neema

*Ritika: her three models, metrics, plots, and error notes from
task2_sentiment/ritika_mukesh_neema/.*

2.C Comparison

Fill this after Ritika's Part 2 is in.

	Sneha	Ritika
Best model	Baseline, macro-F1 0.923	
Acc / macro-F1	0.923 / 0.923	
Takeaway	Mean-pool still leads my two sequence models. McNemar does not separate them.	

Part 3 — CycleGAN, photo to Monet

3.A Sneha Singh

I trained an unpaired CycleGAN. The Kaggle direction is photo to Monet. The generators are mine. I did not use a pretrained model to make the images. Inception and the other nets in the eval script are only for measuring.

Each generator is a ResNet with 9 blocks at 256×256 , reflection padding, instance norm, and a tanh output. Upsampling is nearest-neighbor times two, then a stride-1 convolution, not a transposed convolution. Each discriminator is a PatchGAN trained with least-squares loss. The losses are adversarial, cycle L1 with $\lambda = 10$, and identity at half of that. Real labels are smoothed to 0.9. An image pool of 50 feeds the discriminator. Batch size is 4. I trained 40 epochs at a constant learning rate and 40 more with decay, and each epoch resamples about 800 photos. The shorter Monet loader is cycled so those photos actually get used. Scoring uses all 7,038 photos in the photo-to-Monet direction and all 300 Monet paintings in the other direction.

Training was on a CUDA GPU with mixed precision. It took about 4.3 hours and peaked near 6568 MB. The log records `device=cuda` and does not print the GPU name. `best.pt` is 113 MB, over GitHub's limit, so it is on Drive:

https://drive.google.com/file/d/1TVDCjIGkr5Xt5nrrOmkjhKA3G9ppj-2T/view?usp=share_link

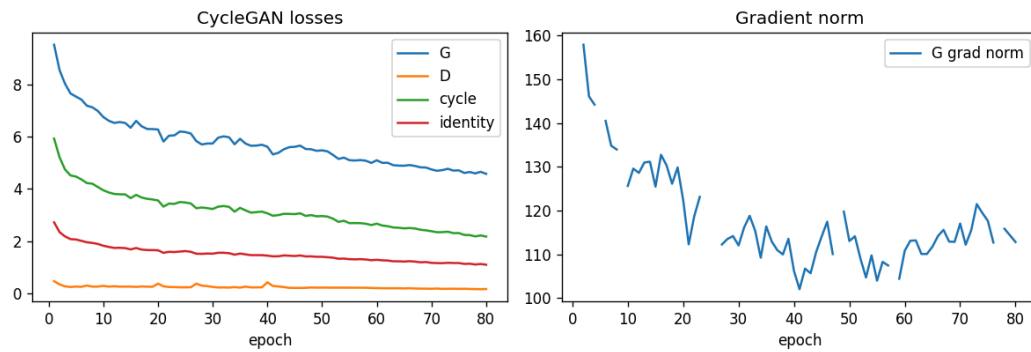
The Monet and photo folders are in one Drive folder:

https://drive.google.com/drive/folders/1BXYfhW8uZ6umK1TZFW8Un62mZVK72L7Y?usp=share_link

Direction	FID	KID	MiFID	LPIPS	Content cosine	Cycle L1
Photo → Monet	89.99	0.018	0.403	0.452	0.532	0.112
Monet → photo	92.23	0.030	0.423	0.365	0.586	0.103

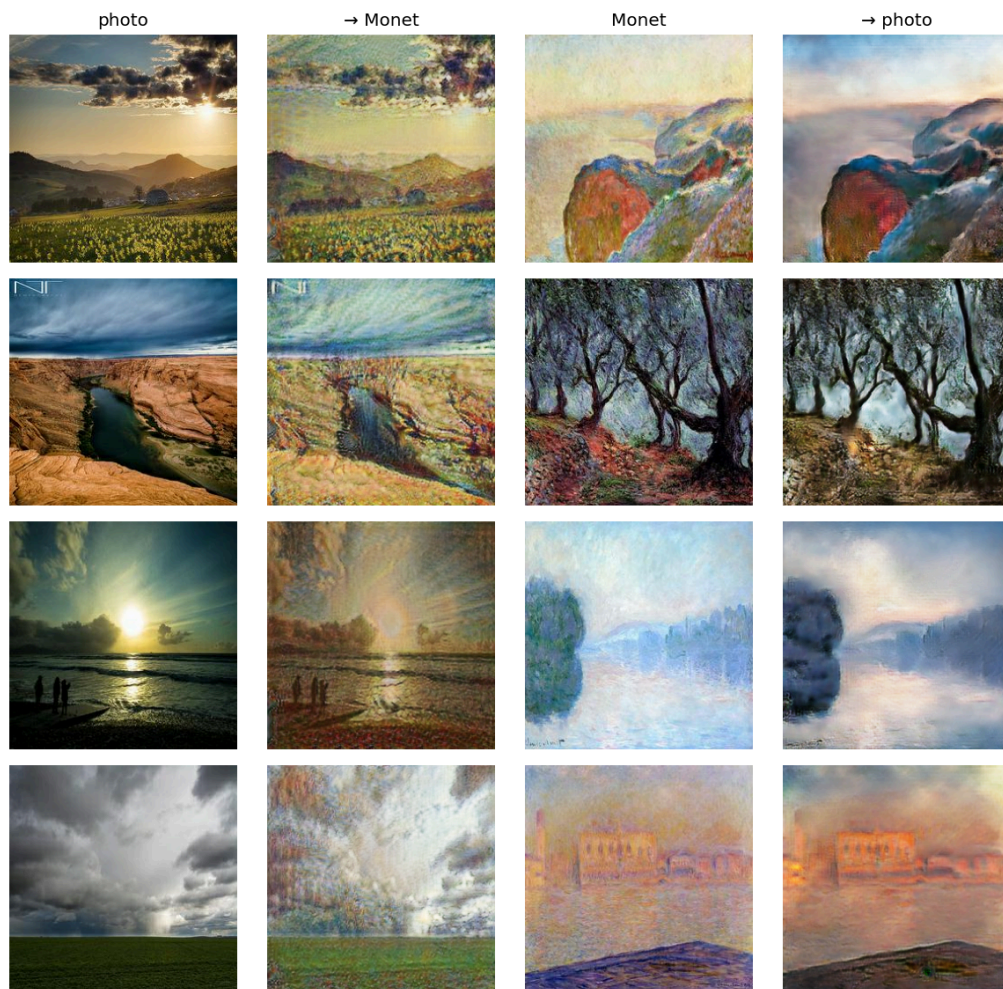
FID is the Fréchet distance between Inception features of the generated images and the real target images. MiFID here is the average cosine distance of those features after an equal subsample. Lower is better on both. My local combined score for the Kaggle direction is $(89.99 + 0.403) / 2 = 45.20$. The same two numbers are in `task3_gan/sneha_singh/submission.csv`. I have not uploaded that file yet, so public score, private score, and rank are still empty.

Losses fell across the 80 epochs. The discriminator loss ended near 0.17, which is low. The generator is not winning that fight, and I think that is why the brush texture gets noisy.



CycleGAN losses and gradient norm

The grid is four photos, the Monet version of each, four real Monets, and the photo version of those.



Photo, generated Monet, real Monet, generated photo

What I see in that grid, and in the 30 images I audited: the layout of the photo usually survives. Skies and water pick up a grainy, repeated dab texture. Some colors wash out. A few Monet-to-photo frames go soft or pick up a dark blob. I did not mark any of my 30 as a copied training Monet. My rough averages on the sheet were style 1.4, content 1.7, artifacts 1.4, on a 0–2 scale where higher means worse. The sheet is

task3_gan/sneha_singh/outputs/human_audit/audit_30.csv. Agreement with Ritika is not computed yet, because she has not scored the same 30.

3.B Ritika Mukesh Neema

Ritika: her CycleGAN, hardware, FID and MiFID, a grid, and her column on the same 30 images. Folder: task3_gan/ritika_mukesh_neema/.

3.C Comparison

Fill this after Ritika's Part 3 is in. Kappa needs both of us on the same 30 files.

	Sneha	Ritika
A2B FID / MiFID	89.99 / 0.403	
Local score (FID + MiFID) / 2	45.20	
Kaggle public / rank	not uploaded yet	
Audit κ		

What is still open

1. Upload my submission.csv to Kaggle for PairProgramming_Team_5, then paste the public score, private score, and rank into this file.
2. Ritika's three sections, and the three comparison tables above.
3. This PDF is the current export. Regenerate it after Ritika's sections and the Kaggle score are filled in.

The plots in this file are the png files already stored under each task folder. They are not copied again into report/.